

CLR PARSING

- CLR parsing refers to Canonical Lookahead.
- CLR stands for Canonical LR parser.
- This method uses large set of items called LR(1) items.

Construction of CLR Parsing Table algorithm

Input: An augmented grammar G'

Output: CLR parsing table.

Method:

- Initially construct set of items $C = \{I_0, I_1, I_2, \dots\}$ where C is a collection of LR(1) items for G .
- Parsing actions are based on each item or state I_i .

Various Actions

- If $A \rightarrow \alpha \cdot a \beta$ is in I_i and $\text{goto}(I_i, a) = I_j$, then create an entry in Action table $\text{Action}[i, a] = \text{shift } j$.
- If $A \rightarrow \alpha \cdot$, a is in I_i , then set in Action table $\text{Action}[i, a]$ to reduce $A \rightarrow \alpha$. Here A should not be in S .
- If $S' \rightarrow S \cdot$ is in I_i , then $\text{Action}[i, \$] = \text{accept}$.
- The goto part of CLR table can be filled as
 - If $\text{goto}(I_i, A) = I_j$ then $\text{goto}[i, A] = j$.

→ In CCR(1), the reduced node will be placed only in the lookahead symbol.

LR(1) Item

LR(1) item is a collection of LR(0) item and a lookahead symbol.

$$LR(1) \text{ item} = LR(0) \text{ item} + \text{lookahead}$$

→ lookahead is used to determine that where we place the final item.

→ lookahead always add $\$$ symbol for the augment production.

Steps involved in CCR(1) Parsing

- Write a context free grammar for the given input string.
- Check ambiguity of the grammar
- Add Augment production in given grammar
- Create canonical collection of LR(1) items
- Draw DFA
- Construct a CCR(1) parsing-table.

Closure(I) CLR/LALR

1. Consider the grammar and compute closure(I) and construct CLR parsing table.

$S \rightarrow CC$

$C \rightarrow cC$

$C \rightarrow d$

Solution

Augmented Grammar

$S' \rightarrow S$

$S \rightarrow CC$

$C \rightarrow cC$

$C \rightarrow d$

Closure(I)

- * Adding $\cdot$ to the R.H.S of the given grammar.
- * Add lookahead symbol for all the productions.
- * Only one lookahead symbol is used to construct parsing table.

$S' \rightarrow \cdot S, \$$

$S \rightarrow \cdot CC, \$$

$C \rightarrow \cdot cC, c/d$

$C \rightarrow \cdot d, c/d$

$\rightarrow$ [Since after $\cdot$, S is present, we have to add S production with $\cdot$ in R.H.S]

To add lookahead symbol for $S \rightarrow \cdot CC$, consider the previous production

$S \rightarrow S, \$$

$S' \rightarrow \cdot S, \$$

$c. cC, a$

$\text{FIRST}(E, \$) = \$$

$A \rightarrow \alpha. B\beta, a$

$\text{FIRST}(\beta, a)$

Compute To consider the present production is $S \rightarrow \cdot CC$

$S \rightarrow E. CC, \$$

$\alpha. B\beta, a$

$\text{FIRST}(C, \$)$

$[c/d, \$]$

look ahead sym

Canonical LR(1) Items or Automaton

$$I_0: S' \rightarrow \cdot S, \$$$

$$S \rightarrow \cdot CC, \$$$

$$C \rightarrow \cdot cC, c/d$$

$$C \rightarrow \cdot d, c/d$$

Perform all possible transition. Transition is moving dot (.) to the next position.

$$Goto(I_0, S)$$

$$I_1: S' \rightarrow S \cdot, \$ \rightarrow [\text{RHS we have only one } S \text{ and for that we moved } (.) \text{ to next position. After moving } (.) \text{ no more non terminals are present so we have only one production}]$$

$$Goto(I_0, C)$$

$$I_2: S \rightarrow C \cdot C, \$$$

$$C \rightarrow \cdot cC, \$$$

$$C \rightarrow \cdot d, \$$$

$$S \rightarrow C \cdot C \epsilon \$ \quad \text{FIRST}(\epsilon, a)$$

$$\alpha \quad B \beta, a \quad \text{FIRST}(\epsilon, \$) = \$$$

$$\text{FIRST}(\beta, a)$$

$$Goto(I_0, c)$$

$$I_0: S \rightarrow C \cdot c, \$$$

Add only the production of C not the lookahead

$$I_3: C \rightarrow c \cdot C, c/d$$

$$C \rightarrow \cdot cC, c/d$$

$$C \rightarrow \cdot d, c/d$$

$$C \rightarrow \cdot cC \quad \text{for this lookahead}$$

$$C \rightarrow \cdot c \quad \text{has to be computed from } S \rightarrow C \cdot C, \$$$

$$Goto(I_0, d)$$

$$I_4: C \rightarrow c \cdot d, c/d$$

$$C \rightarrow c \cdot C, c/d \quad \text{FIRST}(\epsilon, c/d) = c/d$$

$$\alpha \quad B \beta, a \quad = c/d$$

$$\underline{\epsilon}$$

$$Goto(I_2, C)$$

$$I_5: S \rightarrow CC \cdot, \$$$

$$Goto(I_2, c)$$

$$C \rightarrow C \cdot C, \$ \quad \text{FIRST}(\epsilon, \$) = \$$$

$$\alpha \quad B \beta, a \quad = \$$$

$$\underline{\epsilon}$$

$$I_6: C \rightarrow c \cdot C, \$$$

$$C \rightarrow \cdot cC, \$$$

$$C \rightarrow \cdot d, \$$$

$\text{Goto}(I_2, d)$

$I_7: C \rightarrow d., \$$

$\text{Goto}(I_3, c)$

$I_8: C \rightarrow c.c., c/d$

$\text{Goto}(I_3, c)$

$C \rightarrow c.c., c/d$
 $C \rightarrow .cc, c/d$
 $C \rightarrow .d, c/d$

$\text{Goto}(I_3, d)$

$C \rightarrow d., c/d$ 24

$\text{Goto}(I_6, c)$

$I_9: C \rightarrow cc., \$$

$\text{Goto}(I_6, c)$

$C \rightarrow c.c., \$$

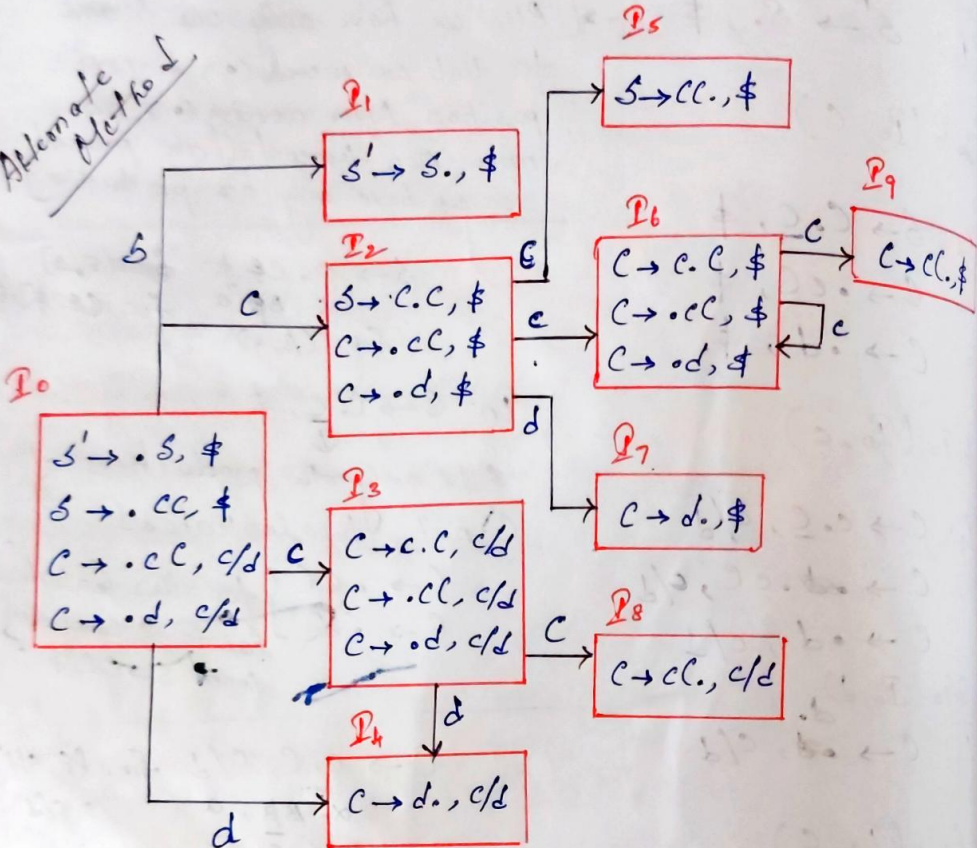
$C \rightarrow .cc, \$$

$C \rightarrow .d, \$$

$\text{Goto}(I_6, d)$

$C \rightarrow d., \$$ 27

Alternate Method



CLR Parsing

add \$

	Stack	Input Buffer	Action
	\$0	<u>c</u> dd \$	Shift S ₃
	\$0c3	<u>d</u> d \$	Shift S ₄
	\$0c3d4	<u>d</u> \$	Reduce r ₃ (c → d)
\$0c3d4 c \$0c3c shift go to	\$0c3c8	<u>d</u> \$	Reduce r ₂ (c → cc)
	\$0c2	<u>d</u> \$	Shift S ₇
\$0c3c8 c \$0c2c8	\$0c2d7	<u>d</u> \$	Reduce r ₃ (c → d)
\$0c2c8 c \$0c2c5	\$0c2c5	<u>d</u> \$	Reduce r ₁ (S → cc)
\$0c2c5 S \$0c1	\$0c1	<u>d</u> \$	Accept

2. Construct CLR parser for the given grammar

$$S \rightarrow L = R \mid R$$

$$L \rightarrow *R \mid id$$

$$R \rightarrow L$$

Also parse the string $id = id$

Solution

Augmented Grammar

$$S' \rightarrow S$$

$$S \rightarrow L = R$$

$$S \rightarrow R$$

$$L \rightarrow *R$$

$$L \rightarrow id$$

$$R \rightarrow L$$

Closure (I)

$$I_0 \left\{ \begin{array}{l} S' \rightarrow \cdot S, \$ \\ S \rightarrow \cdot L = R, \$ \\ L \rightarrow \cdot *R, =/\$ \\ L \rightarrow \cdot id, =/\$ \\ R \rightarrow \cdot L, \$ \end{array} \right.$$

$$S' \rightarrow \cdot S, \$ \\ \alpha \beta, a \\ \text{First}[\epsilon, \$] = \$$$

$$S \rightarrow \cdot L = R, \$ \\ \text{First}(=R, \$)$$

$$R \rightarrow \cdot L, \$$$

Already we have to write the production for L, so we have to write only look ahead

Canonical CR(1) Item

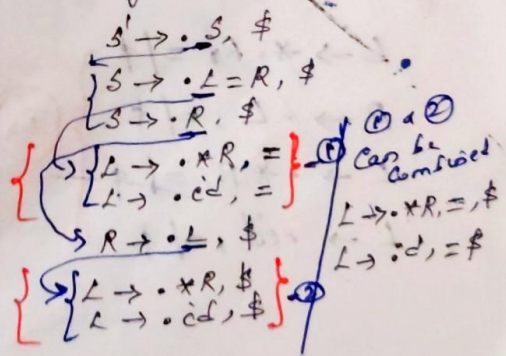
goto(I₀, S)

$$I_1: S' \rightarrow S \cdot, \$$$

goto(I₀, L)

$$I_2: S \rightarrow L \cdot = R, \$$$

$$R \rightarrow L \cdot, \$ \quad (R)$$



goto(I_0, R)

$I_3: S \rightarrow R., \$$ (R)

goto($I_0, *$)

$I_4: L \rightarrow *.R, =/\$$

$R \rightarrow .L, =/\$$

$L \rightarrow .*R, =/\$$

$L \rightarrow .id, =/\$$

goto(I_0, id)

$I_5: L \rightarrow id., =/\$$ (R)

goto($I_0, =$)

$I_6: S \rightarrow L=.R, \$$

$R \rightarrow .L, \$$

$L \rightarrow .*R, \$$

$L \rightarrow .id, \$$

goto(I_4, R)

$I_7: L \rightarrow *.R., =/\$$ (R)

goto(I_4, L)

$I_8: R \rightarrow L., =/\$$ (R)

goto($I_4, *$)

$L \rightarrow *.R, =/\$$

$R \rightarrow .L, =/\$$

$L \rightarrow .*R, =/\$$

$L \rightarrow .id, =/\$$

I_4

goto(I_4, id)

$L \rightarrow id., =/\$$ — I_9

goto(I_6, R)

$I_9: S \rightarrow L=.R., \$$ (R)

goto(I_6, L)

$I_{10}: R \rightarrow L., \$$ (R)

goto($I_6, *$)

$I_{11}: L \rightarrow *.R, \$$

$R \rightarrow .L, \$$

$L \rightarrow .*R, \$$

$L \rightarrow .id, \$$

goto(I_6, id)

$I_{12}: L \rightarrow id., \$$ (R)

goto(I_{11}, R)

$I_{13}: L \rightarrow *.R., \$$ (R)

goto(I_{11}, L)

$R \rightarrow L., \$$ — I_{10}

goto($I_{11}, *$)

$L \rightarrow *.R, \$$

$R \rightarrow .L, \$$

$L \rightarrow .*R, \$$

$L \rightarrow .id, \$$

I_{10}

goto(I_{11}, id)

$L \rightarrow id., \$$ — I_{12}

I_{12}

Construction of CLR parsing table

States	ACTION				Goto		
	id	=	*	#	S	L	R
P_0	S_5		S_4		1	2	3
P_1				Accept			
P_2		S_6		r_5			
P_3				r_2			
P_4	S_5		S_4			8	7
P_5		r_4		r_4			
P_6	S_{12}		S_{11}			10	9
P_7		r_3		r_3			
P_8		r_5		r_5			
P_9				r_1			
P_{10}				r_5			
P_{11}	S_{12}		S_{11}			10	13
P_{12}				r_4			
P_{13}				r_3			

Reduce

- $S \rightarrow L = R$
- $S \rightarrow R$
- $L \rightarrow AR$
- $L \rightarrow id$
- $R \rightarrow L$

$P_3: S \rightarrow R, \#$
 $(3, \#) \rightarrow r_2$
 $\downarrow$
 State Action produces null

$P_5: L \rightarrow id, =/\#$
 $(5, (=/\#)) = r_4$
 $\downarrow$

$P_7: L \rightarrow AR, =/\#$
 $(7, (=/\#)) = r_3$
 $\downarrow$

CLR Parsing

$id = id \$$

State $\swarrow$ $\searrow$ Gob
 $\downarrow$ $L \rightarrow 2$
 $\downarrow$

$\$OL2 = 6id12$
 $\$OL2 = 6L$
 $\$OL2 = 6L10$
 $\$OL2 = 6R9$
 $S \rightarrow L = R$
 $(3 \times 2 = 6)$
 $\$OL3 = 6R9$
 $\$OS$

Stack	Input String	Action
$\$0$	$id = id \$$	Shift S_5
$\$0id5$	$= id \$$	Reduce $r_4 (L \rightarrow id)$
$\$OL2$	$= id \$$	Shift S_6
$\$OL2 = 6$	$id \$$	Shift S_{12}
$\$OL2 = 6id12$	$\$$	Reduce $r_4 (L \rightarrow id)$
$\$OL2 = 6L10$	$\$$	Reduce $r_5 (R \rightarrow L)$
$\$OL2 = 6R9$	$\$$	Reduce r_1 ($S \rightarrow L = R$)
$\$OS1$	$\$$	Accept

(0, id)
 $(S, =)$
 $(2, =)$
 $(6, id)$
 $(12, \$)$
 $(9, \$)$
 $(1, \$)$
 $\downarrow$
State Action

9
 10
 11
 12

3. Construct CLR parser for the grammar

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow id(E)$$

$$T \rightarrow id$$

Solution:

Augmented grammar

$$E' \rightarrow E$$

$$E \rightarrow E + T$$

$$T \rightarrow id(E)$$

$$T \rightarrow id$$

$$E \rightarrow T$$

Closure (E')

~~$$E' \rightarrow \cdot E, \$$$~~

$$E' \rightarrow \cdot E, \$$$

~~$$E' \rightarrow \cdot E + T, \$$$~~

$$E \rightarrow \cdot E + T, \$/+$$

$$E \rightarrow \cdot T, \$/+$$

$$T \rightarrow \cdot id(E), \$/+$$

$$T \rightarrow \cdot id, \$/+$$

$$E' \rightarrow \cdot E, \$$$

$$\textcircled{1} \quad E \rightarrow \cdot E + T, \$$$

$$E \rightarrow \cdot T, \$$$

$$E \rightarrow \cdot E + T, +$$

$$\textcircled{2} \quad E \rightarrow \cdot T, +$$

$$T \rightarrow id(E), \$, +$$

$$T \rightarrow \cdot id, \$, +$$

$R \rightarrow$

$$FIRST(E', \$) = \$$$

$$FIRST(E, +T\$) = +$$

$$FIRST(T, \$) = \$$$

$$FIRST(T, +) = +$$

Combine $\textcircled{1}$ and $\textcircled{2}$

$$E' \rightarrow \cdot E, \$$$

$$E \rightarrow \cdot E + T, \$/+$$

$$E \rightarrow \cdot T, \$/+$$

$$T \rightarrow \cdot id(E), \$/+$$

$$T \rightarrow \cdot id, \$/+$$

Canonical LR(1) Items

$P_0:$

- $E' \rightarrow \cdot E, \$$
- $E \rightarrow \cdot E + T, \$/+$
- $E \rightarrow \cdot T, \$/+$
- $T \rightarrow \cdot id(E), \$/+$
- $T \rightarrow \cdot id, \$/+$

goto(P_0, E)

$P_1:$

- $E' \rightarrow E \cdot, \$$
- $E \rightarrow E \cdot + T, \$/+$

[Check E in the Rets and move dot (.) to next position. No need to find FIRST() in this scenario]

goto(P_0, T)

$P_2:$ $E \rightarrow T \cdot, \$/+$

goto(P_0, id)

$P_3:$

- $T \rightarrow id \cdot (E), \$/+$
- $T \rightarrow id \cdot, \$/+$

goto($P_1, +$)

$P_4:$

- $E \rightarrow E + \cdot T, \$/+$
- $T \rightarrow \cdot id(E), \$/+$
- $T \rightarrow \cdot id, \$/+$

[After . Check for non-terminal here we have T , add ^{only} the production of T not lookahead. After that find the lookahead for $T \rightarrow \cdot id(E)$ from the parent production $E \rightarrow E + T, \$/+$
 $T \rightarrow \cdot id$
 $FIRST(T) = [+, \$/+]$

goto(P_3, id)

~~$T \rightarrow id \cdot (E), \$/+$~~

$P_5:$

- $T \rightarrow id \cdot (E), \$/+$
- $E \rightarrow \cdot E + T,)/+$
- $E \rightarrow \cdot T,)/+$
- $T \rightarrow \cdot id(E),)/+$
- $T \rightarrow \cdot id,)/+$

$FIRST(E) = [), \$/+]$

① $\begin{cases} E \rightarrow \cdot E + T,) \\ E \rightarrow \cdot T,) \end{cases}$

② $\begin{cases} E \rightarrow \cdot E + T, + \\ E \rightarrow \cdot T, + \end{cases}$

$FIRST(E) = [+,)/+]$

$E \rightarrow \cdot E + T,)/+ \quad \text{①} + \text{②}$

$E \rightarrow \cdot T,)/+$

$E \rightarrow \cdot T,)/+ \quad FIRST(T) = [+,)/+]$

goto(P_4, T)

$P_6:$ $E \rightarrow E + T \cdot, \$/+$

goto(I_4 , id)

I_3 : $T \rightarrow id.(E), \$|+$
 $T \rightarrow id. , \$|+$

goto(I_5 , E)

I_7 : $T \rightarrow id(E.), \$|+$
 $E \rightarrow E.+T,)|+$

goto(I_5 , T)

I_8 : $E \rightarrow T. ,)|+$

goto(I_5 , id)

I_9 : $T \rightarrow id.(E),)|+$
 $T \rightarrow id. ,)|+$

goto(I_7 ,)

I_{10} : $T \rightarrow id(E)., \$|+$

goto(I_7 , +)

I_{11} : $E \rightarrow E+.I,)|+$
 $T \rightarrow .id(E), \$|+$
 $T \rightarrow .id ,)|+$

goto(I_{11} , T)

I_{15} : $E \rightarrow E+T.,)|+$

goto(I_9 ,)

I_{12} : $T \rightarrow id.(E),)|+$
 $E \rightarrow .E+T,)|+$
 $E \rightarrow .T ,)|+$
 $T \rightarrow ~~id~~.id(E),)|+$
 $T \rightarrow .id ,)|+$

goto(I_{11} , T)

I_8 : $E \rightarrow E+T.,)|+$

goto(I_{11} , id)

I_9 : $T \rightarrow id.(E),)|+$
 $T \rightarrow id. ,)|+$

goto(I_{12} , E)

I_{13} : $T \rightarrow id(E.),)|+$
 $E \rightarrow E.+T ,)|+$

goto(I_{12} , T)

I_8 : $E \rightarrow T. ,)|+$

goto(I_{12} , id)

I_9 : $T \rightarrow id.(E),)|+$
 $T \rightarrow id. ,)|+$

goto(I_{13} ,)

I_{14} : $T \rightarrow id(E).,)|+$

goto(I_{13} , +)

I_{11} : $E \rightarrow E+T.,)|+$
 $T \rightarrow .id(E),)|+$
 $T \rightarrow .id ,)|+$

CLR Parsing Table

States	ACTION				GOTO		
	+	id	(	)	\$	ET	E
I_0	S_4	S_3			1	2	1
I_1					Accept		
I_2	r_2				r_2		
I_3	r_4		S_5		r_4		
I_4		S_3				6	
I_5		S_9			7	8	7
I_6	r_1				r_1		
I_7	S_{11}			S_{10}			
I_8	r_2		r_2	r_2			
I_9	r_4		S_{12}	r_4			
I_{10}	r_3				r_3		
I_{11}		S_9				15	
I_{12}		S_9				8	13
I_{13}	S_{11}			S_{14}			
I_{14}	r_3			r_3			
I_{15}	r_1			r_1			

Reduce

- $E \rightarrow E + T$
~~$E \rightarrow T$
 $T \rightarrow id(E)$
 $T \rightarrow id$~~
- $E \rightarrow T$
- $T \rightarrow id(E)$
- $T \rightarrow id$

- $I_1: E' \rightarrow E, \$$
- $I_2: E \rightarrow T, (\underline{\$}, +)$
- $I_3: T \rightarrow id, (\underline{\$}, +)$
- $I_6: E \rightarrow E + T, (\underline{\$}, +)$
- $I_8: E \rightarrow T, (\underline{), +)$
- $I_9: T \rightarrow id, (\underline{), +)$
- $I_{10}: T \rightarrow id(E), (\underline{\$}, +)$
- ~~$I_{11}: T \rightarrow id$~~
- $I_{14}: T \rightarrow id(E), (\underline{), +)$
- $I_{15}: E \rightarrow E + T,)/+$